

deelvraag1_dataprep

October 30, 2025

[]:

0.1 Data-preparatie

Dit notebook behandelt de data-preparatie en geeft antwoord op deelvraag 1:

“Hoe kan cellijn-data (mutaties, methylatie, drug-respons) worden verzameld, opgeschoond en geschikt gemaakt voor machine learning?”

Voor dit onderzoek worden vier afzonderlijke datasets gebruikt die afkomstig zijn van het CCLE-project (Cancer Cell Line Encyclopedia). De data beschrijven verschillende moleculaire kenmerken en annotaties van kankercellijnen:

- OmicsSomaticMutations.csv - bevat de somatische mutatieprofielen van de cellijnen.
- CCLE_RRBS_TSS1kb_20181022.txt - bevat DNA-methylatiegegevens gemeten rond de Transcription Start Site (TSS, ± 1 kb). Dit gebied omvat zowel promoterregio's als het eerste exon, die belangrijk zijn voor genregulatie.
- Drug_sensitivity_AUC_(PRISM_Repurposing_Secondary_Screen)_subsetting.csv - bevat therapieresponsgegevens (AUC-waarden) die aangeven hoe gevoelig een cellijn is voor een bepaald medicijn.
- Cell_lines_annotations_20181226.txt - metadata-bestand met annotaties voor elke cellijn, inclusief `depMapID`, weefseltype (`Site_Primary`), histologie en andere klinisch-biologische kenmerken. Dit bestand wordt gebruikt om cellijnen te filteren op weefseltype en om gelijkmatigheid te creëren tussen de verschillende datasets.

Het uiteindelijke doel is om een dataset te creëren waarin elke longkankercellijn zowel moleculaire kenmerken (mutaties en methylatie) als een responswaarde (AUC) bevat, zodat deze kan worden gebruikt als input voor supervised machine learning.

xxx toelichting CpG sites en avg coverage (kwaliteitsmaat) xxx: `CpG_sites_hg19` = genomische coördinaten van CpG-sites in het Hg19-genoom `avg_coverage` = maat voor de betrouwbaarheid van de methylatie-metingen op dat locus

TSS=Transcription Start Site, het beginpunt van een gen waar transcriptie start. ± 1 kb betekent dat de gemeten regio 1 kilobase (1000bp) upstream en downstream van de TSS ligt dus direct rond het begin van het gen, inclusief een stukje voor (promotorgebied) en erna (eerste exon/regio vlakbij TSS)

- Upstream van TSS (~ 1000 bp): dit is meestal de promotor. Methylatie hier kan de transcriptie van het gen onderdrukken.
- Downstream van TSS (~ 1000 bp): dit is vaak het 5' einde van het gen, inclusief exon 1 of de 5' UTR. Methylatie hier kan ook effect hebben op genexpressie. methylatie in deze regio

kan het initiatiefase van transcriptie veranderen, waardoor het gen minder of anders wordt afgelezen

Nadenken over avg_coverage. minimum instellen. te lage waarden weglaten omdat de methyleratiepercentages dan ruis bevatten (in de literatuur 10%) laat ook zien wat de impact is van die drempel.

Stap 1. Data verkennen Doel: inzicht krijgen in de structuur van elk bestand voor het filteren en samenvoegen.

Het is belangrijk om de kolomnamen, rijenstructuur en datatypes te begrijpen. Zo krijg je inzicht in welke kolommen relevant zijn (bijv. cell_line_id, gene, AUC, avg_coverage) en kun je bepalen hoe de datasets aan elkaar gekoppeld kunnen worden.

stappen: Controleer het aantal rijen/kolommen. Bekijk een paar voorbeeldrijen om de structuur te snappen. Noteer de naam van de kolom die de cellijn identificeert (ID). Kijk of de kolomnamen consistent zijn tussen bestanden. Kijk hoeveel unieke cellijnen er per bestand zijn.))

```
[56]: import pandas as pd

# aanmaken variabele met de bestandspad
mutatie_file = "../data/raw/OmicsSomaticMutations.csv"

# inlezen
mutatie_df = pd.read_csv(mutatie_file, low_memory=False) # csv, geen tabs

# verkenning
mutatie_df.info()
print(mutatie_df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 751310 entries, 0 to 751309
```

```
Data columns (total 70 columns):
```

#	Column	Non-Null Count	Dtype
0	Chrom	751310 non-null	object
1	Pos	751310 non-null	int64
2	Ref	751310 non-null	object
3	Alt	751310 non-null	object
4	AF	751310 non-null	float64
5	DP	751310 non-null	int64
6	RefCount	751310 non-null	int64
7	AltCount	751310 non-null	int64
8	GT	751310 non-null	object
9	PS	44923 non-null	float64
10	VariantType	751310 non-null	object
11	VariantInfo	751310 non-null	object
12	DNACChange	750960 non-null	object
13	ProteinChange	726857 non-null	object
14	HugoSymbol	751310 non-null	object

15	Exon	728937 non-null	object
16	Intron	23986 non-null	object
17	EnsemblGeneID	751310 non-null	object
18	EnsemblFeatureID	751310 non-null	object
19	HgncName	734284 non-null	object
20	HgncFamily	531438 non-null	object
21	UniprotID	433949 non-null	object
22	DbsnpRsID	12007 non-null	object
23	GcContent	751310 non-null	float64
24	LofGeneName	102748 non-null	object
25	LofGeneId	102748 non-null	object
26	LofNumberOfTranscriptsInGene	102748 non-null	float64
27	LofPercentOfTranscriptsAffected	102748 non-null	float64
28	NMD	30606 non-null	object
29	MolecularConsequence	25215 non-null	object
30	VepImpact	751310 non-null	object
31	VepBiotype	751310 non-null	object
32	VepHgncID	751294 non-null	object
33	VepExistingVariation	397193 non-null	object
34	VepManeSelect	735516 non-null	object
35	VepENSP	749044 non-null	object
36	VepSwissprot	693449 non-null	object
37	Sift	599796 non-null	object
38	Polyphen	577290 non-null	object
39	GnomadeAF	751310 non-null	float64
40	GnomadgAF	751310 non-null	float64
41	VepClinSig	27442 non-null	object
42	VepSomatic	254466 non-null	object
43	VepPliGeneValue	664121 non-null	float64
44	VepLofTool	562587 non-null	float64
45	OncogeneHighImpact	751310 non-null	bool
46	TumorSuppressorHighImpact	751310 non-null	bool
47	TranscriptLikelyLof	63420 non-null	object
48	BrcalFuncScore	22 non-null	float64
49	CivicID	1381 non-null	float64
50	CivicDescription	728 non-null	object
51	CivicScore	1381 non-null	float64
52	LikelyLoF	751310 non-null	bool
53	HessDriver	751310 non-null	bool
54	HessSignature	2393 non-null	object
55	RevelScore	574655 non-null	float64
56	PharmgkbId	19 non-null	object
57	DidaID	11 non-null	object
58	DidaName	11 non-null	object
59	GwasDisease	257 non-null	object
60	GwasPmID	257 non-null	float64
61	GtexGene	7 non-null	object
62	ProveanPrediction	558282 non-null	object

```

63 AMClass                524520 non-null object
64 AMPathogenicity        524520 non-null float64
65 Rescue                 751310 non-null bool
66 RescueReason           18617 non-null object
67 ModelID                751310 non-null object
68 Hotspot                751310 non-null bool
69 EntrezGeneID           744341 non-null float64
dtypes: bool(6), float64(16), int64(4), object(44)
memory usage: 371.1+ MB

```

	Chrom	Pos	Ref	Alt	AF	DP	RefCount	AltCount	GT	PS	...	\
0	chr1	818203	G	A	0.240	27	21	6	0/1	NaN	...	
1	chr1	851926	G	A	0.158	17	15	2	0/1	NaN	...	
2	chr1	918916	CTGA	C	0.462	51	28	23	0/1	NaN	...	
3	chr1	924510	GC	AA	0.412	35	21	14	0 1	924510.0	...	
4	chr1	924657	C	G	0.437	17	9	8	0/1	NaN	...	

	GwasPmID	GtexGene	ProveanPrediction	AMClass	AMPathogenicity	Rescue	\
0	NaN	NaN		NaN	NaN	NaN	False
1	NaN	NaN		NaN	NaN	NaN	False
2	NaN	NaN		NaN	NaN	NaN	False
3	NaN	NaN		NaN	NaN	NaN	False
4	NaN	NaN		NaN	NaN	NaN	False

	RescueReason	ModelID	Hotspot	EntrezGeneID
0	NaN	ACH-000062	False	400728.0
1	NaN	ACH-001949	False	643837.0
2	NaN	ACH-002059	False	100130417.0
3	NaN	ACH-000402	False	148398.0
4	NaN	ACH-000693	False	148398.0

[5 rows x 70 columns]

```
[119]: mutatie_df['ModelID'].nunique()
```

[119]: 1939

De mutatie-dataset bevat 751310 rijen en 70 kolommen en bestaat uit de volgende datatypes:

- object: tekst/categorieën (bijvoorbeeld genen en mutatietypes)
- float64: decimale waarden
- bool: waar/onwaar waarden (True/False)
- int64: hele getallen (bijvoorbeeld aantallen mutaties)

Belangrijk voor dit onderzoek: ModelID: koppeling tussen datasets HugoSymbol/EnsemblGeneID: gen waarin mutatie voorkomt VariantType: type mutatie (feature voor ML) DNACChange/ProteinChange: optioneel voor interpretatie AF, RefCount, AltCount: betrouwbaarheid / filteren LikelyLoF, OncogeneHighImpact, TumorSuppressorHighImpact: impact op functie/belangrijke features Sift, Polyphen, RevelScore: schadelijkheidsvoorspelling PrimaryDisease/Tissue: filteren op longkankercellijnen

```
[58]: # aanmaken variabele met de bestandpad
methyлатie_file = "../data/raw/CCLE_RRBS_TSS1kb_20181022.txt"

# inlezen
methyлатie_df = pd.read_csv(methyлатie_file, sep='\t', low_memory=False) # txt,
↳ tab seperated

# verkenning
print("Aantal rijen en kolommen:", methyлатie_df.shape) # globale info

# eerste en laatste 5 kolommen bekijken
print("Eerste 5 kolommen:", methyлатie_df.columns[:10].tolist())
print("Laatste 5 kolommen:", methyлатie_df.columns[-10:].tolist())

# datatypes en missende waarden per kolom bekijken
kolom_info_methyлатie = pd.DataFrame({ # aanmaken dataframe (met 3 kolommen)
    'kolom': methyлатie_df.columns,
    'dtype': [methyлатie_df[col].dtype for col in methyлатie_df.columns],
    'n_missing': [methyлатie_df[col].isna().sum() for col in methyлатie_df.
↳ columns] # bepaald totaal aantal missende waarden voor elke kolom
})

# eerste 10 en laatste 10 kolommen tonen als voorbeeld
print(kolom_info_methyлатie.head(10)) # eerste 10 kolommen
print(kolom_info_methyлатie.tail(10)) # laatste 10 kolommen
```

Aantal rijen en kolommen: (21338, 846)

Eerste 5 kolommen: ['locus_id', 'CpG_sites_hg19', 'avg_coverage', 'DMS53_LUNG', 'SW1116_LARGE_INTESTINE', 'P3HR1_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE', 'HUT78_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE', 'UMUC3_URINARY_TRACT', 'HOS_BONE', 'AML193_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE']

Laatste 5 kolommen: ['JHUEM7_ENDOMETRIUM', 'OE21_OESOPHAGUS', 'MOLT3_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE', 'HOP62_LUNG', 'EKVX_LUNG', 'UO31_KIDNEY', 'SF268_CENTRAL_NERVOUS_SYSTEM', 'SF539_CENTRAL_NERVOUS_SYSTEM', 'SNB75_CENTRAL_NERVOUS_SYSTEM', 'HOP92_LUNG']

	kolom	dtype	n_missing
0	locus_id	object	0
1	CpG_sites_hg19	object	1
2	avg_coverage	object	0
3	DMS53_LUNG	object	0
4	SW1116_LARGE_INTESTINE	object	0
5	P3HR1_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	object	0
6	HUT78_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	object	0
7	UMUC3_URINARY_TRACT	object	0
8	HOS_BONE	object	0
9	AML193_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	object	0
	kolom	dtype	n_missing
836	JHUEM7_ENDOMETRIUM	object	0

837	OE21_OESOPHAGUS	object	0
838	MOLT3_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	object	0
839	HOP62_LUNG	object	0
840	EKVX_LUNG	object	0
841	U031_KIDNEY	object	0
842	SF268_CENTRAL_NERVOUS_SYSTEM	object	0
843	SF539_CENTRAL_NERVOUS_SYSTEM	object	0
844	SNB75_CENTRAL_NERVOUS_SYSTEM	object	0
845	HOP92_LUNG	object	0

De methylatie-dataset bevat 21338 rijen en 846 kolommen. Elke rij vertegenwoordigt een specifieke CpG-locus, een locatie op het DNA waar methylatie kan optreden.

De eerste kolommen bevatten metadata: - `locus_id`: unieke identifier van de CpG-locus - `CpG_sites_hg19`: genoompositie van de locus - `avg_coverage`: gemiddelde meetdiepte, een maat voor de betrouwbaarheid van de meting

Alle overige kolommen (vanaf kolom 4) vertegenwoordigen de cellijnen. Wat me opvalt is dat dit bestand geen ACH- of ModelID bevat. In plaats daarvan worden cellijnen geïdentificeerd op basis van hun naam, waarbij de kolomtitel zowel de cellijnnaam als het weefseltype bevat (bijvoorbeeld DMS53_LUNG).

Let op! Om de drie datasets samen te voegen zal een mapping moeten worden gemaakt van deze kolomnamen naar de bijbehorende ACH_ID. Dit zal worden uitgevoerd in stap 5 (samenvoegen).

```
[46]: # aanmaken variabele met de bestandspad
response_file = "../data/raw/
↳Drug_sensitivity_AUC_(PRISM_Repurposing_Secondary_Screen)_subsetting.csv"
# inlezen
response_df = pd.read_csv(response_file, sep=',') # comma seperated

# verkenning
print("Aantal rijen en kolommen:", response_df.shape) # geeft het aantal rijen,
↳en kolommen

# datatypes en missende waarden per kolom bekijken
kolom_info_response = pd.DataFrame({
    'kolom': response_df.columns,
    'dtype': [response_df[col].dtype for col in response_df.columns],
    'n_missing': [response_df[col].isna().sum() for col in response_df.columns]
})

# eerste 20 kolommen tonen als voorbeeld
print(kolom_info_response.head(10))
print(kolom_info_response.tail(10))

print(response_df.iloc[:5,:10])
```

Aantal rijen en kolommen: (480, 1457)

kolom dtype n_missing

0		depmap_id	object	0
1		cell_line_display_name	object	0
2		lineage_1	object	0
3		lineage_2	object	0
4		lineage_3	object	0
5		lineage_6	object	436
6		lineage_4	float64	480
7	8-BROMO-CGMP (BRD:BRD-A00077618-236-07-6)	float64		370
8	NORETYNODREL (BRD:BRD-A00758722-001-04-9)	float64		98
9	PREDNISOLONE-ACETATE (BRD:BRD-A01643550-001-04-9)	float64		432

		kolom	dtype	n_missing
1447	LORLATINIB (BRD:BRD-K99879819-001-02-1)	float64		326
1448	HEXYLRESORCINOL (BRD:BRD-K99946902-001-07-5)	float64		212
1449	BOSULIF (BRD:BRD-K99964838-001-11-9)	float64		220
1450	AMMONIUM-LACTATE (BRD:BRD-M29182745-001-01-4)	float64		433
1451	NEMONAPRIDE (BRD:BRD-M80207679-001-01-5)	float64		446
1452	CROMAKALIM (BRD:BRD-M89827113-001-01-5)	float64		325
1453	EFONIDIPINE-MONOETHANOLATE (BRD:BRD-M92675308-...	float64		164
1454	DICHLOROACETATE (BRD:BRD-M97302542-001-04-4)	float64		234
1455	TYLOXAPOL (BRD:BRD-U25960968-000-01-9)	float64		386
1456	SEVELAMER (BRD:BRD-U45393375-000-01-6)	float64		463

	depmap_id	cell_line_display_name	lineage_1 \
0	ACH-000973	639V	Bladder/Urinary Tract
1	ACH-001016	BECKER	CNS/Brain
2	ACH-000927	BT474	Breast
3	ACH-000288	BT549	Breast
4	ACH-000212	CAL120	Breast

	lineage_2	lineage_3 \
0	Urethral Cancer	Urethral Urothelial Carcinoma
1	Diffuse Glioma	Astrocytoma
2	Invasive Breast Carcinoma	Breast Invasive Ductal Carcinoma
3	Invasive Breast Carcinoma	Breast Invasive Carcinoma, NOS
4	Invasive Breast Carcinoma	Invasive Breast Carcinoma

	lineage_6	lineage_4	8-BROMO-CGMP (BRD:BRD-A00077618-236-07-6) \
0	NaN	NaN	0.887284
1	NaN	NaN	0.932656
2	ER+, PR+, HER2+	NaN	NaN
3	Basal B TNBC	NaN	NaN
4	Basal B TNBC	NaN	NaN

	NORETYNODREL (BRD:BRD-A00758722-001-04-9) \
0	0.870060
1	0.950242
2	0.874661
3	0.974322
4	NaN

	PREDNISOLONE-ACETATE (BRD:BRD-A01643550-001-04-9)
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

De response-dataset bevat 480 rijen, waarbij elke rij een unieke cellijn representeert, en 1457 kolommen. De eerste kolommen bevatten meta-data over de cellijnen:

- depmap_id: unieke ID voor elke cellijn (bijvoorbeeld ACH-000973)
- cell_line_display_name: leesbare naam van de cellijn (bijvoorbeeld 639V of BECKER)
- lineage_1 t/m lineage_6: beschrijving van de biologische afkomst van de cellijn, van breed tot specifiek.

Vanaf kolom 8 bevatten de overige kolommen de drug-response waarden voor verschillende geneesmiddelen, weergegeven als numerieke scores (AUC). Elke cellijn is getest voor meerdere geneesmiddelen, waarbij sommige waarden ontbreken (NaN).

```
[129]: # inlezen van het metadata-bestand
metadata_file = "../data/raw/Cell_lines_annotations_20181226.txt"
metadata_df = pd.read_csv(metadata_file, sep='\t', low_memory=False) # txt, tabu
↳ seperated

# verkenning
print("Aantal rijen en kolommen:", metadata_df.shape) # globale info

print(metadata_df.columns)

# belangrijke kolommen selecteren
cols = ['CCLE_ID', 'depMapID', 'Name', 'Site_Primary',
        'Site_Subtype1', 'Site_Subtype2', 'Site_Subtype3', 'Disease']

# alleen de eerste rijen met deze kolommen tonen
print(metadata_df[cols].head())
```

Aantal rijen en kolommen: (1461, 33)

```
Index(['CCLE_ID', 'depMapID', 'Name', 'Pathology', 'Site_Primary',
       'Site_Subtype1', 'Site_Subtype2', 'Site_Subtype3', 'Histology',
       'Hist_Subtype1', 'Hist_Subtype2', 'Hist_Subtype3', 'Gender',
       'Life_Stage', 'Age', 'Race', 'Geo_Loc', 'inferred_ethnicity',
       'Site_Of_Finding', 'Disease', 'Annotation_Source',
       'Original.Source.of.Cell.Line', 'Characteristics', 'Growth.Medium',
       'Supplements', 'Freezing.Medium', 'Doubling.Time.from.Vendor',
       'Doubling.Time.Calculated.hrs', 'type', 'type_refined',
       'PATHOLOGIST_ANNOTATION', 'mutRate', 'tcga_code'],
      dtype='object')
```

CCLE_ID	depMapID	Name \
---------	----------	--------

0		DMS53_LUNG	ACH-000698	DMS 53
1		SW1116_LARGE_INTESTINE	ACH-000489	SW1116
2		NCIH1694_LUNG	ACH-000431	NCI-H1694
3	P3HR1_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE		ACH-000707	P3HR-1
4	HUT78_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE		ACH-000509	HuT 78

	Site_Primary	Site_Subtype1	Site_Subtype2	\
0	lung	NS	NS	
1	large_intestine	colon	NS	
2	lung	NS	NS	
3	haematopoietic_and_lymphoid_tissue	NS	NS	
4	haematopoietic_and_lymphoid_tissue	skin	NS	

	Site_Subtype3	Disease
0	NS	lung_cancer
1	NS	NaN
2	NS	NaN
3	NS	Burkitts lymphoma
4	NS	sezary_syndrome

beschrijving output metadata aantal rijen en kolomme, de kolommen en de eerste 5 regels van de relevante kolommen die nodig zijn ...

Stap 2. Filteren op relevante data Doel: alleen de biologische en analyse relevante mutaties behouden, zodat de dataset overzichtelijk blijft en rekentijd wordt bespaard.

Mutatiebestand

Acties: 1. filteren op cellijntype: alleen cellijnen van het weefseltype “lung” worden behouden. 2. filteren op relevante mutaties: niet alle mutaties zijn biologisch significant (veel varianten zijn neutraal of technisch artefact). Daarom worden alleen functionele mutaties behouden op basis van de volgende kolommen: - LikelyLoF (verlies-van-functie mutaties) - OncogeneHighImpact (activerende mutaties in oncogenen) - TumorSuppressorHighImpact (verlies-van-remfunctie in tumorsuppressorgen) - Hotspot (bekende activerende hotspot-mutaties) - HessDriver (mutaties geïdentificeerd als drivers) Een mutatie wordt behouden als één of meer van deze criteria waar zijn. 3. transformatie naar binaire matrix: voor elke cellijn (ModelID) wordt per gen (HugoSymbol) gekeken of er een relevante mutatie aanwezig is. Wanneer een mutatie wordt gevonden dan krijgt deze waarde=1, anders waarde=0. Dit resulteert in een binaire matrix waarin rijen en cellijnen representeren en kolommen genen.

```
[123]: # kolommen waar long kan voorkomen
tissue_cols = ['Site_Primary', 'Site_Subtype1', 'Site_Subtype2',
               ↪ 'Site_Subtype3']

# filteren op long
lung_meta = metadata_df[metadata_df[tissue_cols].apply(lambda x: x.str.
               ↪ contains('lung', case = False, na = False)).any(axis=1)
]
```

```

# depMapIDs van longkankercellijnen ophalen
lung_depmap_ids = lung_meta['depMapID'].tolist()

# filter mutatie_df op longkankercellijnen
lung_mutatie_df = mutatie_df[mutatie_df['ModelID'].isin(lung_depmap_ids)]

# check resultaat
print(lung_mutatie_df.head())
print(f"Aantal mutaties in longkankercellijnen in mutatie_df: {lung_mutatie_df.
↪shape[0]}")

# unieke ModelIDs in gefilterde Dataframe op lung
aantal_lung_cellijnen = lung_mutatie_df['ModelID'].nunique()
print(f"Aantal longkankercellijnen: {aantal_lung_cellijnen}")

```

	Chrom	Pos	Ref	Alt	AF	DP	RefCount	AltCount	GT	PS	...	\
0	chr1	818203	G	A	0.240	27	21	6	0/1	NaN	...	
15	chr1	939091	G	A	0.500	30	15	15	0/1	NaN	...	
17	chr1	939389	A	C	0.250	10	8	2	0/1	NaN	...	
26	chr1	942868	A	C	0.649	46	16	30	0/1	NaN	...	
39	chr1	944763	C	CA	0.247	9	7	2	0/1	NaN	...	

	GwasPmID	GtexGene	ProveanPrediction	AMClass	AMPathogenicity	Rescue	\
0	NaN	NaN		NaN	NaN	NaN	False
15	NaN	NaN	Damaging	NaN	NaN	NaN	False
17	NaN	NaN	Damaging	NaN	NaN	NaN	False
26	NaN	NaN	Neutral	NaN	NaN	NaN	False
39	NaN	NaN		NaN	NaN	NaN	False

	RescueReason	ModelID	Hotspot	EntrezGeneID
0	NaN	ACH-000062	False	400728.0
15	NaN	ACH-000669	False	148398.0
17	NaN	ACH-000769	False	148398.0
26	NaN	ACH-000840	False	148398.0
39	NaN	ACH-000769	False	26155.0

[5 rows x 70 columns]

Aantal mutaties in longkankercellijnen in mutatie_df: 80117

Aantal longkankercellijnen: 174

```

[124]: # filter alleen biologisch relevante mutaties
filtered = lung_mutatie_df[
    lung_mutatie_df['LikelyLoF'] |
    lung_mutatie_df['OncogeneHighImpact'] |
    lung_mutatie_df['TumorSuppressorHighImpact'] |
    lung_mutatie_df['Hotspot'] |
    lung_mutatie_df['HessDriver']

```

```

]

# Maak binaire matrix per gen per celijn
mut_matrix = (
    filtered
    .assign(mutated=1)
    .pivot_table(index='ModelID', columns='HugoSymbol', values='mutated',
    ↪fill_value=0)
    .astype(int)
)

print(mut_matrix.shape)
print(mut_matrix.iloc[:, :5].head)

```

```

(174, 7358)
<bound method NDFrame.head of HugoSymbol  A1CF  A2M  A2ML1  A4GALT  AAAS
ModelID
ACH-000012      0    0      0      0      0
ACH-000015      0    0      0      0      0
ACH-000021      0    0      0      0      0
ACH-000029      0    0      0      0      0
ACH-000030      0    0      0      0      0
...
ACH-001049      0    0      0      0      0
ACH-001075      0    0      0      0      0
ACH-001113      0    0      0      0      0
ACH-001138      0    0      0      0      0
ACH-001233      0    0      0      0      0

```

```
[174 rows x 5 columns]>
```

Output: - de matrix bevat 174 rijen (één per longkankercelijn) en 7358 kolommen (één per gen).
 - elke rij komt overeen met een unieke cellijn (ModelID) die in de metadata geclassificeerd is als afkomstig van longweefsel. - elke kolom stelt een gen voor (HugoSymbol) dat ten minste één van de longkankercellijnen een relevante mutatie bevat. - de celwaarden zijn binair: - 1 = het gen bevat minstens één relevante mutatie in die cellijn. - 0 = het gen bevat geen relevante mutatie in die cellijn.

```
[ ]: Methylatie bestand filteren
xxx
```

```
[100]: # check kolomnamen
print(methylatie_df.columns[:20])
```

```

Index(['locus_id', 'CpG_sites_hg19', 'avg_coverage', 'DMS53_LUNG',
      'SW1116_LARGE_INTESTINE', 'P3HR1_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE',
      'HUT78_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE', 'UMUC3_URINARY_TRACT',
      'HOS_BONE', 'AML193_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE', 'RVH421_SKIN',
      'NCIH1184_LUNG', 'HCC2157_BREAST', 'NCIH2227_LUNG', 'SNU449_LIVER',

```

```
'NCIH28_PLEURA', 'OV56_OVARY', 'RMUGS_OVARY',
'LN229_CENTRAL_NERVOUS_SYSTEM',
'P31FUJ_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE'],
dtype='object')
```

Probleem: Methyatiebestand heeft geen DepMap- of ACH-ID's, maar gebrubikt kolomnamen die bestaan uit cellijnnaam + weefseltype (zoals DMS53_LUNG, SW1116_LARGE_INTESTINE). Elke kolom is een cellijn en elke rij is een metylatie-feature (CpG-site).

In plaats van filteren op rijen, moeten de kolommen worden geselecteerd die bij longkanker horen (dus de kolommen waarvan de naam "_LUNG" bevat).

```
[125]: # filter kolommen waar "LUNG" in voorkomt
lung_columns = [col for col in methyatie_df.columns if 'lung' in col.lower()]

# maak dataframe met longkanker cellijnen + locusinfo
lung_methyatie_df = methyatie_df[['locus_id', 'CpG_sites_hg19',
    ↪ 'avg_coverage'] + lung_columns]

print(f"Aantal longkanker kolommen: {len(lung_columns)}")
print(lung_methyatie_df.shape)
print(lung_methyatie_df.head)
```

Aantal longkanker kolommen: 153

(21338, 156)

<bound method NDFrame.head of locus_id \

```
0      SGIP1_1_66998638_66999638
1      SGIP1_1_66998251_66999251
2      AZIN2_1_33545713_33546713
3      AZIN2_1_33546778_33547778
4      AGBL4_1_50489626_50490626
...
21333   SYCE3_22_51001381_51002381
21334   RABL2B_22_51222110_51223110
21335  RPL23AP82_22_51194513_51195513
21336  RPL23AP82_22_51221156_51222156
21337                                     NA_NA
```

	CpG_sites_hg19	avg_coverage \
0	1:66998970;1:66998973;1:66998993;1:66999404;1:...	25.00
1	1:66998970;1:66998973;1:66998993	8.27
2	1:33546151;1:33546209;1:33546210;1:33546385;1:...	326.58
3	1:33546783;1:33546788;1:33546795;1:33546797;1:...	480.54
4	1:50489632;1:50489641;1:50489671;1:50489677;1:...	263.36
...	...	...
21333	22:51001382;22:51001388;22:51001398;22:5100139...	359.68
21334	22:51222730	6.38
21335	22:51195083;22:51195404;22:51195430	62.10

21336	22:51222010;22:51222012;22:51222018;22:5122202...	179.18
21337	NaN	NA

	DMS53_LUNG	NCIH1184_LUNG	NCIH2227_LUNG	RERFLCAD2_LUNG	NCIH2347_LUNG	\
0	0.00000	0.11864	0.04545	NaN	0.25000	
1	0.00000	0.22579	NaN	NaN	NaN	
2	0.00729	0.22553	0.04560	0.14577	0.11487	
3	0.22276	0.10735	0.03326	0.09589	0.21004	
4	0.00000	0.07346	0.01246	0.00531	0.07422	
...	...	...	...	...	...	
21333	0.00000	0.00000	0.00722	0.03099	0.00650	
21334	0.00000	0.00000	0.00000	0.00000	0.00000	
21335	0.00000	0.00000	0.01150	0.00000	0.00000	
21336	0.00000	0.00000	0.00000	0.00000	0.00000	
21337	NA	NA	NA	NA	NA	

	NCIH2087_LUNG	NCIH2085_LUNG	...	NCIH1944_LUNG	NCIH2405_LUNG	BEN_LUNG	\
0	0.86441	1.00000	...	0.10344	NaN	0.90907	
1	0.97871	NaN	...	0.11110	NaN	NaN	
2	0.17708	0.15413	...	0.02693	0.00697	0.07692	
3	0.35571	0.03926	...	0.04961	0.00380	0.01200	
4	0.60335	0.00000	...	0.00130	0.11082	0.01485	
...	...	...	...	...	...	...	
21333	0.04710	0.00359	...	0.07877	0.01422	0.00882	
21334	0.00000	0.00000	...	0.00000	0.00000	0.00000	
21335	0.02777	0.20237	...	0.03737	0.00992	0.01083	
21336	0.11571	0.00000	...	0.00952	0.00000	0.00000	
21337	NA	NA	...	NA	NA	NA	

	NCIH727_LUNG	HCC2935_LUNG	LCLC103H_LUNG	LCLC97TM1_LUNG	HOP62_LUNG	\
0	NaN	NaN	NaN	1.00000	NA	
1	NaN	NaN	NaN	NaN	NA	
2	0.05825	0.24294	0.01045	0.03746	NA	
3	0.12764	0.36441	0.24371	0.23342	NA	
4	0.10371	0.01515	0.05079	0.86448	NA	
...	...	...	...	...	...	
21333	0.00581	0.00000	0.08452	0.13561	NA	
21334	0.00000	0.00000	0.00000	0.07140	NA	
21335	0.00660	0.00000	0.00533	0.00187	NA	
21336	0.00000	0.00000	0.00000	0.00155	NA	
21337	NA	NA	NA	NA	NA	

	EKVX_LUNG	HOP92_LUNG
0	NA	0.66667
1	NA	0.66667
2	NA	0.00046
3	NA	0.05159
4	NA	0.54553

```
...      ...      ...
21333      NA      0.02364
21334      NA      0.00000
21335      NA      0.00000
21336      NA      0.00000
21337      NA      NA
```

[21338 rows x 156 columns]>

uitleg over dat aantal features te hoog is, hoe verder filteren

filteren op kwaliteit (avg coverage)

```
[111]: # maak kopie
lung_methylatie_df = lung_methylatie_df.copy()

# converteer avg_coverage naar numeriek
lung_methylatie_df['avg_coverage'] = pd.
    ↪to_numeric(lung_methylatie_df['avg_coverage'], errors='coerce')

#verwijder rijen met NaN in avg_coverage
lung_methylatie_df = lung_methylatie_df.dropna(subset=['avg_coverage'])

# filter loci met avg_coverage >=10
filtered_10_df = lung_methylatie_df[lung_methylatie_df['avg_coverage']>=10]

# check hoe veel loci overblijven
print(f"Loci voor filtering op avg coverage: {len(lung_methylatie_df)}")
print(f"Loci na filtering op avg coverage: {len(filtered_10_df)}")
```

Loci voor filtering op avg coverage: 21337

Loci na filtering op avg coverage: 20898

[]:

uitleg: voor ML zijn dit nog steeds te veel features (zeker voor 100-200 cellijnen). nu variantie-filtering: Bij DNA-methylatiegegevens worden duizende CpG-loci gemeten. Veel van deze GpC loci variëren nauwelijks tussen cellijnen. Deze lage-variantie loci voegen weinig voorspellende waarde toe voor machine learning en verhogen alleen de ruis en het aantal features.

[]:

```
[116]: import numpy

# Maak een kopie zodat we veilig bewerkingen kunnen doen
filtered_10_df = filtered_10_df.copy()

# Lijst van kolommen met alleen longkankercellijn-methylatie
```

```

methylation_cols = [col for col in filtered_10_df.columns if col not in
↳ ['locus_id', 'CpG_sites_hg19', 'avg_coverage']]

# Converteer alle waarden naar numeriek, foutieve waarden worden NaN
filtered_10_df[methylation_cols] = filtered_10_df[methylation_cols].apply(pd.
↳ to_numeric, errors='coerce')

# Optioneel: verwijder loci waar ALLE cellijnen NaN zijn
filtered_10_df = filtered_10_df.dropna(subset=methylation_cols, how='all')

# Bereken de standaardafwijking per locus over alle longkanker cellijnen
std_per_locus = filtered_10_df[methylation_cols].std(axis=1)

# Stel drempel in: loci met std >= 0.05 behouden
threshold = 0.05
filtered_var_df = filtered_10_df[std_per_locus >= threshold]

print(f"Aantal loci voor variantie-filtering: {len(filtered_10_df)}")
print(f"Aantal loci na variantie-filtering: {len(filtered_var_df)}")

```

Aantal loci voor variantie-filtering: 20898

Aantal loci na variantie-filtering: 12434

```

[117]: # Bekijk de eerste 5 rijen en 10 kolommen van de dataset
print(filtered_var_df.iloc[:5, :10])
# Bekijk de eerste 5 rijen en alle methylatiekolommen
methylation_cols = [col for col in filtered_var_df.columns if col not in
↳ ['locus_id', 'CpG_sites_hg19', 'avg_coverage']]
print(filtered_var_df[methylation_cols].head())

```

	locus_id \				
0	SGIP1_1_66998638_66999638				
2	AZIN2_1_33545713_33546713				
3	AZIN2_1_33546778_33547778				
4	AGBL4_1_50489626_50490626				
6	CLIC4_1_25070759_25071759				

	CpG_sites_hg19	avg_coverage \			
0	1:66998970;1:66998973;1:66998993;1:66999404;1:...	25.00			
2	1:33546151;1:33546209;1:33546210;1:33546385;1:...	326.58			
3	1:33546783;1:33546788;1:33546795;1:33546797;1:...	480.54			
4	1:50489632;1:50489641;1:50489671;1:50489677;1:...	263.36			
6	1:25070902;1:25070906;1:25071609;1:25071616;1:...	304.38			

	DMS53_LUNG	NCIH1184_LUNG	NCIH2227_LUNG	RERFLCAD2_LUNG	NCIH2347_LUNG \
0	0.00000	0.11864	0.04545	NaN	0.25000
2	0.00729	0.22553	0.04560	0.14577	0.11487
3	0.22276	0.10735	0.03326	0.09589	0.21004

4	0.00000	0.07346	0.01246	0.00531	0.07422
6	0.12261	0.05042	0.12625	0.09678	0.04676

	NCIH2087_LUNG	NCIH2085_LUNG
0	0.86441	1.00000
2	0.17708	0.15413
3	0.35571	0.03926
4	0.60335	0.00000
6	0.14154	0.03906

	DMS53_LUNG	NCIH1184_LUNG	NCIH2227_LUNG	RERFLCAD2_LUNG	NCIH2347_LUNG \
0	0.00000	0.11864	0.04545	NaN	0.25000
2	0.00729	0.22553	0.04560	0.14577	0.11487
3	0.22276	0.10735	0.03326	0.09589	0.21004
4	0.00000	0.07346	0.01246	0.00531	0.07422
6	0.12261	0.05042	0.12625	0.09678	0.04676

	NCIH2087_LUNG	NCIH2085_LUNG	NCIH510_LUNG	NCIH2066_LUNG	NCIH1341_LUNG \
0	0.86441	1.00000	0.00000	NaN	NaN
2	0.17708	0.15413	0.05894	0.04225	0.16046
3	0.35571	0.03926	0.28478	0.12770	0.46391
4	0.60335	0.00000	0.22090	0.01145	0.69029
6	0.14154	0.03906	0.00000	0.00000	0.12690

...	NCIH1944_LUNG	NCIH2405_LUNG	BEN_LUNG	NCIH727_LUNG	HCC2935_LUNG \
0 ...	0.10344	NaN	0.90907	NaN	NaN
2 ...	0.02693	0.00697	0.07692	0.05825	0.24294
3 ...	0.04961	0.00380	0.01200	0.12764	0.36441
4 ...	0.00130	0.11082	0.01485	0.10371	0.01515
6 ...	0.12196	0.04292	0.12834	0.05154	0.00000

	LCLC103H_LUNG	LCLC97TM1_LUNG	HOP62_LUNG	EKVX_LUNG	HOP92_LUNG
0	NaN	1.00000	NaN	NaN	0.66667
2	0.01045	0.03746	NaN	NaN	0.00046
3	0.24371	0.23342	NaN	NaN	0.05159
4	0.05079	0.86448	NaN	NaN	0.54553
6	0.06977	0.11667	NaN	NaN	0.12500

[5 rows x 153 columns]

[]:

Mapping en transponeren van methylatie DataFrame

In deze stap voeren we twee acties uit: 1. mapping van kolommen naar ACH-IDs 2. Transponseren van de DataFrame

Hierbij wordt ook een snelle controle uitgevoerd om te zien hoeveel cellijnen in methylatie en mutatie voorkomen en hoeveel in beide datasets overlappen. Dit is een check waarbij nog geen cellijnen worden verwijderd, omdat er later nog responswaarden bij komen

```

[133]: # Actie 1. mapping
# alleen de methylatiekolommen van de overgebleven loci
methylation_cols = [col for col in filtered_var_df.columns
                     if col not in ['locus_id', 'CpG_sites_hg19', 'avg_coverage']]

# maak mapping: CCLE_ID (kolomnaam) -> depMapID
ccle2depmap = dict(zip(metadata_df['CCLE_ID'], metadata_df['depMapID']))

# map kolommen en filter de cellijnen die geen mapping hebben eruit
new_cols = []
valid_cols = []
for col in methylation_cols:
    depmap_id = ccle2depmap.get(col)
    if depmap_id: # alleen houden als mapping bestaat
        new_cols.append(depmap_id)
        valid_cols.append(col)

# pas mapping toe op DataFrame
meth_mapped_df = filtered_var_df[['locus_id', 'CpG_sites_hg19', 'avg_coverage'] +
    ↪ valid_cols].copy()
meth_mapped_df.columns = ['locus_id', 'CpG_sites_hg19', 'avg_coverage'] + new_cols

print(f"Aantal cellijnen na mapping: {len(new_cols)}")
print(meth_mapped_df.head())

# Actie 2. transponeren
# transposeer methylatie zodat cellijnen rijen zijn en loci kolommen
meth_T = meth_mapped_df.
    ↪ drop(columns=['locus_id', 'CpG_sites_hg19', 'avg_coverage']).T
meth_T.index.name = 'ModelID' # ACH-ID als index, net als in mut_matrix
# Zorg dat de index in beide datasets de ACH-ID is
meth_ids = set(meth_T.index) # methylatie cellijnen
mut_ids = set(mut_matrix.index) # mutatie cellijnen

# aantal unieke cellijnen in elk bestand
print(f"Aantal cellijnen in methylatie: {len(meth_ids)}")
print(f"Aantal cellijnen in mutaties: {len(mut_ids)}")

# aantal cellijnen die in beide bestanden voorkomen
common_ids = meth_ids.intersection(mut_ids)
print(f"Aantal cellijnen in beide bestanden: {len(common_ids)}")

# aantal cellijnen die alleen in methylatie zitten
only_meth = meth_ids - mut_ids
print(f"Cellijnen alleen in methylatie: {len(only_meth)}")

# aantal cellijnen die alleen in mutaties zitten

```

```
only_mut = mut_ids - meth_ids
print(f"Cellijnen alleen in mutaties: {len(only_mut)}")
```

Aantal cellijnen na mapping: 153

```
locus_id \
0 SGIP1_1_66998638_66999638
2 AZIN2_1_33545713_33546713
3 AZIN2_1_33546778_33547778
4 AGBL4_1_50489626_50490626
6 CLIC4_1_25070759_25071759
```

```
CpG_sites_hg19 avg_coverage \
0 1:66998970;1:66998973;1:66998993;1:66999404;1:... 25.00
2 1:33546151;1:33546209;1:33546210;1:33546385;1:... 326.58
3 1:33546783;1:33546788;1:33546795;1:33546797;1:... 480.54
4 1:50489632;1:50489641;1:50489671;1:50489677;1:... 263.36
6 1:25070902;1:25070906;1:25071609;1:25071616;1:... 304.38
```

```
ACH-000698 ACH-000523 ACH-000610 ACH-000774 ACH-000875 ACH-000841 \
0 0.00000 0.11864 0.04545 NaN 0.25000 0.86441
2 0.00729 0.22553 0.04560 0.14577 0.11487 0.17708
3 0.22276 0.10735 0.03326 0.09589 0.21004 0.35571
4 0.00000 0.07346 0.01246 0.00531 0.07422 0.60335
6 0.12261 0.05042 0.12625 0.09678 0.04676 0.14154
```

```
ACH-000451 ... ACH-000414 ACH-000121 ACH-000603 ACH-000775 \
0 1.00000 ... 0.10344 NaN 0.90907 NaN
2 0.15413 ... 0.02693 0.00697 0.07692 0.05825
3 0.03926 ... 0.04961 0.00380 0.01200 0.12764
4 0.00000 ... 0.00130 0.11082 0.01485 0.10371
6 0.03906 ... 0.12196 0.04292 0.12834 0.05154
```

```
ACH-000150 ACH-000627 ACH-000596 ACH-000861 ACH-000706 ACH-000825
0 NaN NaN 1.00000 NaN NaN 0.66667
2 0.24294 0.01045 0.03746 NaN NaN 0.00046
3 0.36441 0.24371 0.23342 NaN NaN 0.05159
4 0.01515 0.05079 0.86448 NaN NaN 0.54553
6 0.00000 0.06977 0.11667 NaN NaN 0.12500
```

[5 rows x 156 columns]

Aantal cellijnen in methylatie: 153

Aantal cellijnen in mutaties: 174

Aantal cellijnen in beide bestanden: 149

Cellijnen alleen in methylatie: 4

Cellijnen alleen in mutaties: 25

Stap 3. Opschonen Doel: ervoor zorgen dat alle bestanden dezelfde structuur en sleutel-IDs gebruiken.

De kolomnamen moeten worden gestandaardiseerd. Dit houdt in dat alledrie de bestanden dezelfde sleutel-IDs moeten krijgen: `cell_line_id`. Ook moeten dubbele rijen of niet-unieke cellijnen moeten worden verwijderd. Er moet gecontroleerd worden op ontbrekende waarden (NaN) in de belangrijke kolommen. Dit is belangrijk om fouten te voorkomen bij het samenvoegen.

xxx `avc_cov` in deze stap? > Waarden lager dan 10 worden verwijderd, omdat deze metingen onvoldoende dekking hebben om betrouwbare methylatiegegevens te geven. Dit voorkomt dat technische ruis invloed heeft op het model.

Stap 4. Transformatie van features Doel: - mutatiegegevens omzetten naar binaire variabelen (0=mutatie, 1=mutatie). - methylatiegegevens reduceren/aggregeren (gemiddelde pakken?)

Machine-learningmodellen werken met numerieke inputs; binaire encoding wordt veelgebruikt.

Methylatie heeft vaak duizenden features wat kan leiden tot overfitting? **meer uitleg nodig, bron**

Wanneer er meerdere AUC-metingen zijn per cellijn moet het gemiddelde worden berekend.

[126]:

```
Aantal overlappende cellijnen: 0
Overlappende cellijnen: []
```

Stap 5. Samenvoegen Doel: het creëren van één tabel waarin elke rij één cellijn is, met alle relevante features en bijbehorende responswaarde.

De drie datasets worden samengevoegd op de gemeenschappelijke sleutel `cell_line_id`.

Er wordt gebruik gemaakt van een **inner join** om ontbrekende waarden in de uiteindelijke tabel te voorkomen. Alleen cellijnen waarvan zowel moleculaire data (mutaties, methylatie) als responsdata beschikbaar zijn, worden behouden. Dit is een belangrijke eis voor supervised learning.

check ook hoeveel cellijnen verloren gaan door die join.

[]:

Stap 6. Controle op ontbrekende waarden Doel: controleren of de uiteindelijke dataset compleet genoeg is.

Cellijnen met onbrekende AUC moeten worden verwijderd omdat machine-learningalgoritmen niet kunnen omgaan met ontbrekende waarden.

[]:

Stap 7. Preprocessing voor machine learning Doel: de data omzetten naar numerieke vorm, schalen en voorbereiden voor modellering.

hier start scikit-learn Pandas is vooral geschikt voor het inlezen, bewerken en samenvoegen van dataframes. Scikit-learn is gespecialiseerd in preprocessing.

xxx verwijst naar scikit-learn tutorial?

[]: